


# JAVA PROGRAMMING LAB (BCS-154)

## B.Tech CSE 1st SEMESTER

**SUBMITTED TO:**

**MR. ABHAY GUPTA SIR**

**SUBMITTED BY:**

**AAROHI SHUKLA**

INVERTIS UNIVERSITY BAREILLY

# INDEX


| SN. | PROGRAM NAME | PAGE | DATE | SIGNATURE |
|---|---|---|---|---|
| 1 | Hello World, Variables and Arithmetic | 3 | 26/10/2026 | |
| 2 | Decisions and Loops with Numbers | 5 | 26/10/2026 | |
| 3 | Star and Number Patterns | 7 | 26/10/2026 | |
| 4 | Writing Functions | 9 | 28/10/2026 | |
| 5 | Arrays: Traversal and Counting | 11 | 28/10/2026 | |
| 6 | Arrays: Reverse and Duplicates | 13 | 28/10/2026 | |
| 7 | Counting Steps and Complexity | 15 | 30/10/2026 | |
| 8 | Matrices: Transpose and Row Sums | 17 | 30/10/2026 | |
| 9 | Matrices: Rows and Columns | 19 | 30/10/2026 | |
| 10 | Switch and ArrayList | 21 | 02/11/2026 | |
| 11 | Bubble Sort and Selection Sort | 23 | 02/11/2026 | |
| 12 | Insertion Sort | 25 | 02/11/2026 | |
| 13 | Strings: Methods and Palindromes | 27 | 04/11/2026 | |
| 14 | Characters: Counting and Positions | 29 | 04/11/2026 | |
| 15 | StringBuilder Methods | 31 | 04/11/2026 | |

# EXPERIMENT NO. 1

## Hello World, Variables and Arithmetic

### 1. Experiment Title:

Hello World, Variables and Arithmetic

### 2. Aim:

Print text, store values in variables and perform calculations using arithmetic operators.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

Java programs are written inside classes, and execution begins from the main() method. Variables store values of specific data types such as int, double, char and String. Arithmetic operators include +, -, *, / and %. The final keyword declares a constant whose value cannot be reassigned.

### 5. Algorithm / Procedure:

1. Start the program and define the Main class.
2. Print the two introductory messages on separate lines.
3. Declare the required variables and print each value with its label.
4. Declare two integers and calculate their sum, difference, product, quotient and remainder.
5. Declare PI as a constant and calculate the circle's area and perimeter.
6. Display all results and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Print messages
        System.out.println("Hello World");
        System.out.println("Welcome to Java");

        // Task 2: Variables
        int roll = 101;
        double percentage = 84.5;
        char section = 'B';
        String name = "Asha";

        System.out.println("Roll: " + roll);
        System.out.println("Percentage: " + percentage);
        System.out.println("Section: " + section);
        System.out.println("Name: " + name);

        // Task 3: Arithmetic operators
        int a = 17, b = 5;
        System.out.println("Sum: " + (a + b));
        System.out.println("Difference: " + (a - b));
        System.out.println("Product: " + (a * b));
        System.out.println("Quotient: " + (a / b));
        System.out.println("Remainder: " + (a % b));

        // Task 4: Constant and circle calculations
        final double PI = 3.14;
        double radius = 7;
        System.out.println("Area: " + (PI * radius * radius));
        System.out.println("Perimeter: " + (2 * PI * radius));
    }
}
```

### 7. Input:

No keyboard input is required. Values are initialized in the program.

## 8. Output:

```
Hello World
Welcome to Java
Roll: 101
Percentage: 84.5
Section: B
Name: Asha
Sum: 22
Difference: 12
Product: 85
Quotient: 3
Remainder: 2
Area: 153.86
Perimeter: 43.96
```

## Time and Space Complexity:

Time: O(1) — a fixed number of statements. Space: O(1) — a fixed number of variables.

## 9. Result:

The program successfully prints messages, displays variable values, performs arithmetic operations and calculates the area and perimeter of a circle.

# EXPERIMENT NO. 2

## Decisions and Loops with Numbers

### 1. Experiment Title:

Decisions and Loops with Numbers

### 2. Aim:

Use if-else, else-if and loops to make decisions and perform repeated calculations.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

Decision-making statements control program flow. An if-else statement selects between two alternatives, while an else-if ladder checks multiple conditions. The modulus operator helps test divisibility. Loops repeat statements; a for loop is useful when the number of iterations is known. A leap year is divisible by 400, or divisible by 4 but not by 100.

### 5. Algorithm / Procedure:

1. Start and declare the required integer values.
2. Check whether 27 is divisible by 2 and print whether it is even or odd.
3. Use an else-if ladder to assign a grade to 83 marks.
4. Check the leap-year condition for 2024.
5. Use a for loop to calculate the factorial of 5.
6. Display the results and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Even or odd
        int n = 27;
        if (n % 2 == 0)
            System.out.println(n + " is even");
        else
            System.out.println(n + " is odd");

        // Task 2: Grade using else-if
        int marks = 83;
        if (marks >= 90)
            System.out.println("Grade: A");
        else if (marks >= 75)
            System.out.println("Grade: B");
        else
            System.out.println("Grade: C");

        // Task 3: Leap year
        int year = 2024;
        if ((year % 4 == 0 && year % 100 != 0) ||
             year % 400 == 0)
            System.out.println("Leap year");
        else
            System.out.println("Not a leap year");

        // Task 4: Factorial
        int factorial = 1;
        for (int i = 1; i <= 5; i++) {
            factorial *= i;
        }
        System.out.println("Factorial of 5 is " + factorial);
    }
}
```

### 7. Input:

No keyboard input is required. The program uses n = 27, marks = 83 and year = 2024.

### 8. Output:

```
27 is odd
Grade: B
Leap year
Factorial of 5 is 120
```

### Time and Space Complexity:

Even/odd, grade and leap-year checks take O(1) time each. Factorial calculation takes O(n) time and O(1) auxiliary space; here n = 5.

### 9. Result:

The program successfully demonstrates conditional statements, leap-year checking and factorial calculation using a loop.

# EXPERIMENT NO. 3

## Star and Number Patterns

### 1. Experiment Title:

Star and Number Patterns

### 2. Aim:

Use nested loops to print square, triangular and inverted patterns of stars and numbers.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

A nested loop is a loop placed inside another loop. The outer loop generally controls the rows, while the inner loop controls the number of symbols printed in each row. Changing the inner-loop limit produces different patterns, such as squares, right triangles and inverted triangles.

### 5. Algorithm / Procedure:

1. Start the program.
2. Use two nested loops to print four rows of four stars.
3. Use an inner loop from 1 to the current row number to print a right triangle.
4. Print numbers from 1 to the current row number to create a number triangle.
5. Use a decreasing inner-loop limit to print an inverted star triangle.
6. Move to a new line after each row and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Square of stars
        System.out.println("Square:");
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 4; j++)
                System.out.print("*");
            System.out.println();
        }

        // Task 2: Right triangle
        System.out.println("Right triangle:");
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++)
                System.out.print("*");
            System.out.println();
        }

        // Task 3: Number triangle
        System.out.println("Number triangle:");
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++)
                System.out.print(j + " ");
            System.out.println();
        }

        // Task 4: Inverted triangle
        System.out.println("Inverted triangle:");
        for (int i = 4; i >= 1; i--) {
            for (int j = 1; j <= i; j++)
                System.out.print("*");
            System.out.println();
        }
    }
}
```

### 7. Input:

No keyboard input is required. Each pattern uses 4 rows.

### 8. Output:

```
Square:
****
****
****
****
Right triangle:
*
**
***
****
Number triangle:
1
1 2
1 2 3
1 2 3 4
Inverted triangle:
****
***
**
*
```

### Time and Space Complexity:

For an n × n square, time is $O(n^2)$. The triangular patterns also take $O(n^2)$ time. Auxiliary space is O(1), excluding printed output.

### 9. Result:

The program successfully generates four different patterns using nested for loops.

# EXPERIMENT NO. 4

## Writing Functions

### 1. Experiment Title:

Writing Functions

### 2. Aim:

Create and call user-defined functions with parameters and return values to calculate factorial, combinations, permutations and maximum values.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

A method is a reusable block of code that performs a specific task. A method may accept parameters and return a value. Factorial is the product of all positive integers up to n. Combination is calculated as nCr = n! / (r! × (n-r)!), while permutation is nPr = n! / (n-r)!.

### 5. Algorithm / Procedure:

1. Define a fact(n) method that multiplies integers from 1 to n.
2. Define nCr(n, r) using the factorial method.
3. Define nPr(n, r) using the factorial method.
4. Define maxOf(a, b) to return the larger integer.
5. Call the methods from main() and print their results.
6. Stop.

### 6. Program:

```
public class Main {
    static long fact(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++)
            result *= i;
        return result;
    }

    static long nCr(int n, int r) {
        return fact(n) / (fact(r) * fact(n - r));
    }

    static long nPr(int n, int r) {
        return fact(n) / fact(n - r);
    }

    static int maxOf(int a, int b) {
        return (a > b) ? a : b;
    }

    public static void main(String[] args) {
        System.out.println("5C2 = " + nCr(5, 2));
        System.out.println("5P2 = " + nPr(5, 2));
        System.out.println("Larger: " + maxOf(14, 9));
    }
}
```

### 7. Input:

No keyboard input is required. Method calls use n = 5, r = 2 and maxOf(14, 9).

### 8. Output:

```
5C2 = 10
5P2 = 20
```

```
Larger: 14
```

### Time and Space Complexity:

fact(n) takes O(n) time and O(1) auxiliary space. nCr and nPr make a constant number of factorial calls, so their time complexity is O(n). maxOf() takes O(1) time and O(1) space.

### 9. Result:

The program successfully demonstrates method definition, parameters, return values and method reuse.

# EXPERIMENT NO. 5

## Arrays: Traversal and Counting

### 1. Experiment Title:

Arrays: Traversal and Counting

### 2. Aim:

Create arrays, traverse their elements, calculate their sum and largest value, and count elements satisfying a condition.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

An array stores multiple values of the same data type in a fixed-size sequence. Each element is accessed using an index, starting from 0. The length property gives the number of elements. Array traversal means visiting each element, commonly with a for loop.

### 5. Algorithm / Procedure:

1. Declare an array containing 3, 4 and 5.
2. Traverse it using a loop and print each index and value.
3. Declare the second array and initialize sum and largest using its first element.
4. Traverse the array to update the sum and largest value.
5. Declare the third array and count values greater than 10.
6. Print all results and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Print index and element
        int[] arr = {3, 4, 5};
        for (int i = 0; i < arr.length; i++)
            System.out.println("Index " + i + ": " + arr[i]);

        // Task 2: Sum and largest value
        int[] numbers = {12, 45, 7, 33, 21};
        int sum = 0;
        int largest = numbers[0];

        for (int value : numbers) {
            sum += value;
            if (value > largest)
                largest = value;
        }
        System.out.println("Sum: " + sum);
        System.out.println("Largest: " + largest);

        // Task 3: Count values greater than 10
        int[] values = {5, 12, 2, 14, 7};
        int count = 0;
        for (int value : values) {
            if (value > 10)
                count++;
        }
        System.out.println("Count greater than 10: " + count);
    }
}
```

### 7. Input:

No keyboard input is required. Arrays are initialized with the values specified in the assignment.

## 8. Output:

```
Index 0: 3
Index 1: 4
Index 2: 5
Sum: 118
Largest: 45
Count greater than 10: 2
```

## Time and Space Complexity:

Each array traversal takes O(n) time. Auxiliary space is O(1), as only a fixed number of variables are used.

## 9. Result:

The program successfully traverses arrays, calculates the sum and maximum element, and counts values greater than 10.

# EXPERIMENT NO. 6

## Arrays: Reverse and Duplicates

### 1. Experiment Title:

Arrays: Reverse and Duplicates

### 2. Aim:

Print array elements in reverse order, find the smallest value and its index, and detect duplicate values.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

An array can be traversed from the last index to the first to display its elements in reverse order. The smallest element can be found by maintaining a minimum value and its index during traversal. Nested loops can compare pairs of elements to identify duplicates.

### 5. Algorithm / Procedure:

1. Declare the first array and traverse it from the last index to index 0.
2. Declare the second array and initialize the smallest value and index with its first element.
3. Traverse the second array and update the smallest value and index when a smaller value is found.
4. Declare the third array.
5. Use nested loops to compare each element with the elements after it.
6. Print the repeated value if found; otherwise print the no-duplicate message.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Reverse order
        int[] first = {10, 20, 30, 40, 50};
        for (int i = first.length - 1; i >= 0; i--)
            System.out.println(first[i]);

        // Task 2: Smallest value and its index
        int[] second = {2, 5, 1, 8, 3, 4};
        int smallest = second[0];
        int index = 0;

        for (int i = 1; i < second.length; i++) {
            if (second[i] < smallest) {
                smallest = second[i];
                index = i;
            }
        }
        System.out.println("Smallest: " + smallest +
                           " at index " + index);

        // Task 3: Find a repeated value
        int[] third = {4, 9, 2, 9, 7};
        boolean found = false;

        for (int i = 0; i < third.length && !found; i++) {
            for (int j = i + 1; j < third.length; j++) {
                if (third[i] == third[j]) {
                    System.out.println("Repeated value: " + third[i]);
                    found = true;
                    break;
                }
            }
        }
        if (!found)
            System.out.println("No repeated value");
    }
```

```
}
```

### 7. Input:

No keyboard input is required. Arrays are initialized with the values specified in the assignment.

### 8. Output:

```
50
40
30
20
10
Smallest: 1 at index 2
Repeated value: 9
```

### Time and Space Complexity:

Reverse traversal and minimum search take O(n) time. Duplicate detection with nested loops takes $O(n^2)$ time in the worst case. Auxiliary space is O(1).

### 9. Result:

The program successfully displays an array in reverse, finds the minimum element and its index, and detects a repeated value.

# EXPERIMENT NO. 7

## Counting Steps and Complexity

### 1. Experiment Title:

Counting Steps and Complexity

### 2. Aim:

Count loop iterations and understand the time complexity of single loops, nested loops and repeated halving.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

Time complexity describes how the number of operations grows as input size increases. A single loop running n times has O(n) time complexity. Two nested loops that each run n times have $O(n^2)$ time complexity. Repeatedly halving a value takes O(log n) time because the value is divided by 2 in each iteration.

### 5. Algorithm / Procedure:

1. Initialize a step counter to zero.
2. Run a single loop from 1 to 10 and increment the counter in each iteration.
3. Reset the counter and run two nested loops from 1 to 10, incrementing it in the inner loop.
4. Set n to 1024 and repeatedly divide it by 2 until it becomes 1.
5. Count each division and print all three counts.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: One loop - O(n)
        int steps = 0;
        for (int i = 1; i <= 10; i++)
            steps++;
        System.out.println("Steps: " + steps);

        // Task 2: Two nested loops - O(n^2)
        steps = 0;
        for (int i = 1; i <= 10; i++) {
            for (int j = 1; j <= 10; j++)
                steps++;
        }
        System.out.println("Steps: " + steps);

        // Task 3: Repeated halving - O(log n)
        int n = 1024;
        int divisions = 0;
        while (n > 1) {
            n = n / 2;
            divisions++;
        }
        System.out.println("Halving steps: " + divisions);
    }
}
```

### 7. Input:

No keyboard input is required. The loop limits are 10 and the halving example starts with n = 1024.

### 8. Output:

```
Steps: 10
Steps: 100
```

```
Halving steps: 10
```

### Time and Space Complexity:

Task 1: O(n) time. Task 2: O($n^2$) time. Task 3: O(log n) time. Auxiliary space for each task is O(1).

### 9. Result:

The program successfully counts iterations and demonstrates linear, quadratic and logarithmic time complexity.

# EXPERIMENT NO. 8

## Matrices: Transpose and Row Sums

### 1. Experiment Title:

Matrices: Transpose and Row Sums

### 2. Aim:

Create and traverse two-dimensional arrays, calculate a matrix transpose and find the sum of each row.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

A matrix is represented in Java using a two-dimensional array. It contains rows and columns, and each element is accessed using two indices: matrix[row][column]. Nested loops are used to traverse matrix elements. The transpose of a matrix is formed by interchanging its rows and columns.

### 5. Algorithm / Procedure:

1. Create a 3 × 4 matrix and fill it with values from 1 to 12 using nested loops.
2. Print the matrix row by row.
3. Initialize a 3 × 3 matrix with values 1 to 9.
4. Create its transpose by assigning transpose[j][i] = matrix[i][j].
5. Print the transpose.
6. Traverse the original 3 × 3 matrix and calculate each row's sum.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Create and print a 3 x 4 matrix
        int[][] m = new int[3][4];
        int value = 1;

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++)
                m[i][j] = value++;
        }

        System.out.println("Matrix:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++)
                System.out.print(m[i][j] + " ");
            System.out.println();
        }

        int[][] a = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Task 2: Transpose
        int[][] transpose = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++)
                transpose[j][i] = a[i][j];
        }

        System.out.println("Transpose:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++)
                System.out.print(transpose[i][j] + " ");
            System.out.println();
        }
```

```
        // Task 3: Sum of each row
        for (int i = 0; i < 3; i++) {
            int sum = 0;
            for (int j = 0; j < 3; j++)
                sum += a[i][j];
            System.out.println("Row " + (i + 1) + " sum: " + sum);
        }
    }
}
```

### 7. Input:

No keyboard input is required. The matrices are initialized in the program.

### 8. Output:

```
Matrix:
1 2 3 4
5 6 7 8
9 10 11 12
Transpose:
1 4 7
2 5 8
3 6 9
Row 1 sum: 6
Row 2 sum: 15
Row 3 sum: 24
```

### Time and Space Complexity:

Filling, printing and transposing an r × c matrix take O(r × c) time. Row-sum calculation takes O(r × c) time. The transpose requires O(r × c) additional space.

### 9. Result:

The program successfully creates and prints matrices, calculates a transpose and finds the sum of every row.

# EXPERIMENT NO. 9

## Matrices: Rows and Columns

### 1. Experiment Title:

Matrices: Rows and Columns

### 2. Aim:

Access specific matrix rows, calculate column sums and count matrix values satisfying a condition.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

A two-dimensional array stores values in rows and columns. The first index represents the row and the second index represents the column. A nested loop can traverse all elements. To calculate a column sum, the column index is kept fixed while the row index changes.

### 5. Algorithm / Procedure:

1. Initialize a 3 × 3 matrix with values 1 to 9.
2. Print the first row using row index 0.
3. Print the last row using row index 2.
4. For each column, add the elements from all rows and print the column sum.
5. Traverse all elements and count values greater than 5.
6. Print the count and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        int[][] a = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Task 1: First and last row
        System.out.print("First row: ");
        for (int j = 0; j < 3; j++)
            System.out.print(a[0][j] + " ");
        System.out.println();

        System.out.print("Last row: ");
        for (int j = 0; j < 3; j++)
            System.out.print(a[2][j] + " ");
        System.out.println();

        // Task 2: Sum of each column
        for (int j = 0; j < 3; j++) {
            int sum = 0;
            for (int i = 0; i < 3; i++)
                sum += a[i][j];
            System.out.println("Column " + (j + 1) +
                               " sum: " + sum);
        }

        // Task 3: Count values greater than 5
        int count = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (a[i][j] > 5)
                    count++;
            }
        }
        System.out.println("Values greater than 5: " + count);
```

```
    }
}
```

### 7. Input:

No keyboard input is required. The 3 × 3 matrix is initialized in the program.

### 8. Output:

```
First row: 1 2 3
Last row: 7 8 9
Column 1 sum: 12
Column 2 sum: 15
Column 3 sum: 18
Values greater than 5: 4
```

### Time and Space Complexity:

Row printing takes O(c) time. Column sums and counting take O(r × c) time. Auxiliary space is O(1).

### 9. Result:

The program successfully accesses the first and last rows, calculates column sums and counts values greater than 5.

# EXPERIMENT NO. 10

## Switch and ArrayList

### 1. Experiment Title:

Switch and ArrayList

### 2. Aim:

Use a switch statement to select a day and perform basic operations on an ArrayList.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

The switch statement selects one block of code from multiple alternatives based on an expression's value. ArrayList is a resizable collection from the java.util package. It supports operations such as add(), get(), set(), remove() and size(). ArrayList uses zero-based indexing.

### 5. Algorithm / Procedure:

1. Store 3 in the day variable.
2. Use switch to print the corresponding day name, with a default case for invalid values.
3. Create an ArrayList of Integer and add 10, 20, 30 and 40.
4. Print the list, its size and the element at index 2.
5. Create another list with the same values.
6. Change the element at index 1 to 25 and remove the element at index 3.
7. Print the updated list and its size.

### 6. Program:

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        // Task 1: Switch statement
        int day = 3;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Invalid day");
        }

        // Task 2: Add and access ArrayList elements
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);

        System.out.println(list);
        System.out.println("Size: " + list.size());
        System.out.println("Item at index 2: " + list.get(2));

        // Task 3: Update and remove elements
        ArrayList<Integer> updated = new ArrayList<>();
        updated.add(10);
        updated.add(20);
```

```
        updated.add(30);
        updated.add(40);

        updated.set(1, 25);
        updated.remove(3);

        System.out.println(updated);
        System.out.println("Size: " + updated.size());
    }
}
```

### 7. Input:

No keyboard input is required. The day value is 3 and the ArrayList values are initialized in the program.

### 8. Output:

```
Wednesday
[10, 20, 30, 40]
Size: 4
Item at index 2: 30
[10, 25, 30]
Size: 3
```

### Time and Space Complexity:

A switch with a fixed number of cases takes O(1) time for this example. ArrayList get() and set() take O(1) time; add() at the end is amortized O(1); removing by index may take O(n) due to shifting. Auxiliary storage for the list is O(n).

### 9. Result:

The program successfully demonstrates switch-case selection and ArrayList insertion, access, update, removal and size operations.

# EXPERIMENT NO. 11

## Bubble Sort and Selection Sort

### 1. Experiment Title:

Bubble Sort and Selection Sort

### 2. Aim:

Sort arrays in increasing order using bubble sort and selection sort, and count swaps performed by bubble sort.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element moves to its correct position. Selection sort repeatedly selects the smallest element from the unsorted portion and places it at the beginning of that portion. Both algorithms have $O(n^2)$ average and worst-case time complexity.

### 5. Algorithm / Procedure:

1. Initialize the array 5, 1, 4, 2, 8.
2. Apply bubble sort by comparing adjacent elements and swapping when required.
3. Print the sorted array.
4. Initialize the array 64, 25, 12, 22, 11.
5. Apply selection sort by finding the minimum element in each unsorted portion and swapping it into position.
6. Print the sorted array.
7. Run bubble sort on 5, 1, 4, 2, 8 again, count every swap and print the count.

### 6. Program:

```
import java.util.Arrays;

public class Main {
    static int bubbleSort(int[] a) {
        int swaps = 0;
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                    swaps++;
                }
            }
        }
        return swaps;
    }

    static void selectionSort(int[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIndex])
                    minIndex = j;
            }
            int temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
```

```
    public static void main(String[] args) {
        // Task 1: Bubble sort
        int[] first = {5, 1, 4, 2, 8};
        bubbleSort(first);
        System.out.println("Sorted: " + Arrays.toString(first));

        // Task 2: Selection sort
        int[] second = {64, 25, 12, 22, 11};
        selectionSort(second);
        System.out.println("Sorted: " + Arrays.toString(second));

        // Task 3: Count bubble sort swaps
        int[] third = {5, 1, 4, 2, 8};
        int swaps = bubbleSort(third);
        System.out.println("Swaps: " + swaps);
    }
}
```

### 7. Input:

No keyboard input is required. The arrays are initialized as specified in the assignment.

### 8. Output:

```
Sorted: [1, 2, 4, 5, 8]
Sorted: [11, 12, 22, 25, 64]
Swaps: 4
```

### Time and Space Complexity:

Bubble sort: $O(n^2)$ average/worst-case time and O(1) auxiliary space. Selection sort: $O(n^2)$ time and O(1) auxiliary space. The bubble sort swap count for the given array is 4.

### 9. Result:

The program successfully sorts arrays using bubble sort and selection sort and counts the swaps performed by bubble sort.

# EXPERIMENT NO. 12

## Insertion Sort

### 1. Experiment Title:

Insertion Sort

### 2. Aim:

Sort arrays in increasing order using insertion sort and count the number of shifts.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

Insertion sort builds a sorted portion of an array one element at a time. It takes the next element as a key, shifts larger elements one position to the right and inserts the key into its correct position. It is efficient for small or nearly sorted arrays. Its best-case time complexity is O(n), while its average and worst-case time complexity is $O(n^2)$.

### 5. Algorithm / Procedure:

1. Define an insertionSort method that accepts an integer array.
2. Start from the second element and store it as key.
3. Compare the key with elements to its left.
4. Shift every larger element one position to the right and count each shift.
5. Insert the key at its correct position.
6. Repeat until the array is sorted.
7. Apply the method to all three arrays and print the sorted arrays and shift count.

### 6. Program:

```
import java.util.Arrays;

public class Main {
    static int insertionSort(int[] a) {
        int shifts = 0;

        for (int i = 1; i < a.length; i++) {
            int key = a[i];
            int j = i - 1;

            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j];
                shifts++;
                j--;
            }
            a[j + 1] = key;
        }
        return shifts;
    }

    public static void main(String[] args) {
        // Task 1
        int[] first = {7, 3, 5, 1};
        insertionSort(first);
        System.out.println("Sorted: " + Arrays.toString(first));

        // Task 2
        int[] second = {14, 3, 9, 21, 6};
        insertionSort(second);
        System.out.println("Sorted: " + Arrays.toString(second));

        // Task 3: Count shifts
        int[] third = {6, 5, 4, 3, 2, 1};
        int shifts = insertionSort(third);
```

```
        System.out.println("Shifts: " + shifts);
    }
}
```

### 7. Input:

No keyboard input is required. The arrays are initialized as specified in the assignment.

### 8. Output:

```
Sorted: [1, 3, 5, 7]
Sorted: [3, 6, 9, 14, 21]
Shifts: 15
```

### Time and Space Complexity:

Best-case time: O(n), when the array is already sorted. Average and worst-case time: $O(n^2)$. Auxiliary space: O(1). The reverse-sorted array of six elements requires 15 shifts.

### 9. Result:

The program successfully sorts all three arrays using insertion sort and counts the shifts for the reverse-sorted array.

# EXPERIMENT NO. 13

## Strings: Methods and Palindromes

### 1. Experiment Title:

Strings: Methods and Palindromes

### 2. Aim:

Use String methods, reverse a word, check whether a word is a palindrome and print substrings.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

String is a class in Java used to represent a sequence of characters. Common methods include length(), charAt(), indexOf(), toUpperCase() and substring(). Strings are immutable, which means their contents cannot be changed after creation. A palindrome is a word that reads the same forward and backward.

### 5. Algorithm / Procedure:

1. Store the word Programming and print its length, first occurrence position of g and uppercase form.
2. Store Java and build its reverse by reading characters from the end to the beginning using charAt().
3. Store racecar and compare characters from both ends to check whether it is a palindrome.
4. Store abc and use nested loops with substring() to print all substrings in the required order.
5. Display the results and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: String methods
        String word = "Programming";
        System.out.println("Length: " + word.length());
        System.out.println("Position of g: " + word.indexOf('g'));
        System.out.println("Upper case: " + word.toUpperCase());

        // Task 2: Reverse a word using charAt()
        String text = "Java";
        String reversed = "";
        for (int i = text.length() - 1; i >= 0; i--)
            reversed += text.charAt(i);
        System.out.println("Reversed: " + reversed);

        // Task 3: Palindrome check
        String palindrome = "racecar";
        boolean isPalindrome = true;
        for (int i = 0; i < palindrome.length() / 2; i++) {
            if (palindrome.charAt(i) !=
                palindrome.charAt(palindrome.length() - 1 - i)) {
                isPalindrome = false;
                break;
            }
        }

        if (isPalindrome)
            System.out.println(palindrome + " is a palindrome");
        else
            System.out.println(palindrome + " is not a palindrome");

        // Task 4: Print substrings
        String s = "abc";
        for (int i = 0; i < s.length(); i++) {
            for (int j = i + 1; j <= s.length(); j++)
                System.out.println(s.substring(i, j));
        }
    }
```

```
}
```

### 7. Input:

No keyboard input is required. The strings are initialized in the program.

### 8. Output:

```
Length: 11
Position of g: 3
Upper case: PROGRAMMING
Reversed: avaJ
racecar is a palindrome
a
ab
abc
b
bc
c
```

### Time and Space Complexity:

String length, indexOf and uppercase conversion take O(n) time in general. Reversal using repeated String concatenation can take $O(n^2)$ time. Palindrome checking takes O(n) time and O(1) auxiliary space. Printing all substrings involves $O(n^2)$ substrings (and $O(n^3)$ total character output in the general case).

### 9. Result:

The program successfully demonstrates String methods, reversal, palindrome checking and substring generation.

# EXPERIMENT NO. 14

## Characters: Counting and Positions

### 1. Experiment Title:

Characters: Counting and Positions

### 2. Aim:

Use charAt() to count vowels, count occurrences of a character and print each character with its index.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

The charAt(index) method returns the character at a specified position in a String. A loop can visit every character using indices from 0 to length() - 1. Conditional statements can be used to count vowels or occurrences of a particular character. Java String indexing starts at 0.

### 5. Algorithm / Procedure:

1. Store programming in a String variable s.
2. Traverse the string using charAt() and count characters that are a, e, i, o or u.
3. Traverse the string again and count occurrences of g.
4. Store hello in another String.
5. Print every character with its index, starting from 0.
6. Display the counts and stop.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: Count vowels
        String s = "programming";
        int vowels = 0;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' ||
                ch == 'o' || ch == 'u') {
                vowels++;
            }
        }
        System.out.println("Vowels: " + vowels);

        // Task 2: Count occurrences of g
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == 'g')
                count++;
        }
        System.out.println("Count of g: " + count);

        // Task 3: Print character positions
        String word = "hello";
        for (int i = 0; i < word.length(); i++)
            System.out.println(i + ": " + word.charAt(i));
    }
}
```

### 7. Input:

No keyboard input is required. The strings are initialized in the program.

### 8. Output:

```
Vowels: 3
```

```
Count of g: 2
0: h
1: e
2: l
3: l
4: o
```

### Time and Space Complexity:

Each character-counting traversal takes O(n) time. Printing character positions takes O(n) time. Auxiliary space is O(1).

### 9. Result:

The program successfully counts vowels and occurrences of g and prints each character with its zero-based index.

# EXPERIMENT NO. 15

## StringBuilder Methods

### 1. Experiment Title:

StringBuilder Methods

### 2. Aim:

Use StringBuilder to append, insert and delete characters from a mutable sequence of text.

### 3. Software / Tools Required:

Java Development Kit (JDK), Visual Studio Code or any Java IDE.

### 4. Theory:

StringBuilder is a Java class used to create and modify mutable sequences of characters. Unlike String, a StringBuilder object can be changed without creating a new object for every modification. append() adds text at the end, insert() adds text at a specified index, deleteCharAt() removes a character at a specified index and length() returns the number of characters.

### 5. Algorithm / Procedure:

1. Create a StringBuilder containing Hello.
2. Append the text World with a leading space and print the text and its length.
3. Create an empty StringBuilder.
4. Append numbers 1 to 5, each followed by a comma.
5. Remove the last comma using deleteCharAt().
6. Create a StringBuilder containing Java.
7. Insert Hello and a space at index 0 and print the result.

### 6. Program:

```
public class Main {
    public static void main(String[] args) {
        // Task 1: append() and length()
        StringBuilder first = new StringBuilder("Hello");
        first.append(" World");
        System.out.println(first);
        System.out.println("Length: " + first.length());

        // Task 2: Append numbers and remove final comma
        StringBuilder second = new StringBuilder();
        for (int i = 1; i <= 5; i++) {
            second.append(i);
            second.append(",");
        }
        second.deleteCharAt(second.length() - 1);
        System.out.println(second);

        // Task 3: insert()
        StringBuilder third = new StringBuilder("Java");
        third.insert(0, "Hello ");
        System.out.println(third);
    }
}
```

### 7. Input:

No keyboard input is required. The initial StringBuilder values are provided in the program.

### 8. Output:

```
Hello World
Length: 11
1,2,3,4,5
```

```
Hello Java
```

### Time and Space Complexity:

Appending, inserting and deleting are generally O(n) in the worst case when characters need to be shifted or capacity changes. The loop appends five values, so this example takes O(1) time for a fixed number of values. Auxiliary space is O(n) for the built text.

### 9. Result:

The program successfully demonstrates append(), length(), deleteCharAt() and insert() using StringBuilder.